

Transformers: Core Components

Simplified study notes. Every component is stated as: what it is → a worked example → the detail that is easy to miss. A full numerical walk-through of one sentence is at the end.

The one-paragraph version

A transformer converts text into vectors, then repeatedly lets every vector **look at every other vector and rewrite itself**, then converts the final vector back into a probability distribution over the next token. Every component below is either plumbing for that loop, or the plain feed-forward network you already know.

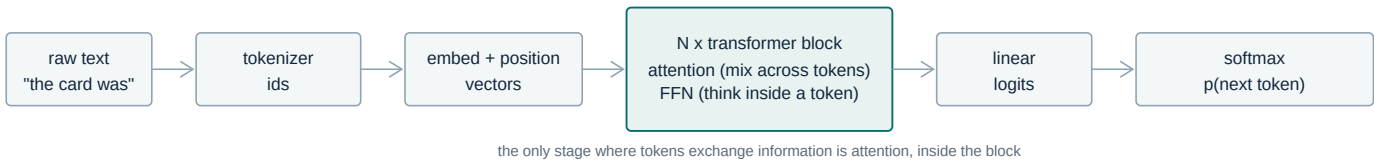


Figure 1 — the whole architecture, end to end.

1 Why attention replaced recurrence

Three ways to let position 1 influence position 50, compared by **maximum path length** (how many steps information must travel) and by how much can run in parallel.

Architecture	Max path length	Parallel over positions?	Consequence
RNN	$O(n)$	No — step t needs step $t - 1$	Long dependencies decay; training is sequential
CNN	$O(n/k)$	Yes	Needs many stacked layers to widen the receptive field
Self-attention	$O(1)$	Yes	Any two positions connect in one hop

EASY TO MISS

Shorter paths are the headline, but **parallel training throughput** is the reason transformers actually won. An RNN cannot use a whole GPU on one sentence; a transformer processes all positions of a sequence in a single matrix multiply.

2 Tokenization

Splitting raw text into pieces from a fixed vocabulary (roughly 30k–200k entries) and mapping each piece to an integer id. Modern tokenizers are sub-word (BPE / WordPiece / SentencePiece), not word-level.

EXAMPLE

un expected ly → ids [359, 21075, 1029] . Common words become one token; rare words shatter into fragments. A start marker such as <sos> and an end marker <eos> are added around the

sequence.

Why sub-word: a word-level vocabulary explodes and can never spell an unseen word; a character-level vocabulary never runs out but makes sequences several times longer, and attention cost grows with the square of length.

EASY TO MISS

The tokenizer is trained **separately and frozen before** the model, so it silently fixes the model's blind spots. The model never sees letters, only fragments — which is why letter-counting inside a word is unreliable. Numbers split inconsistently (1234 may become 12|34 in one place and 1|234 in another), which is why raw arithmetic is fragile. Text in scripts under-represented in the tokenizer costs two to three times more tokens for the same sentence.

3 Input embeddings

A lookup table of shape **vocab** $\times$ d_{model} : token id in, dense vector out. Nothing clever happens here — it is one row of a large learned matrix. d_{model} was 512 originally; modern models use 4096 and above.

MENTAL MODEL

This is the token's **context-free** meaning. At this point the word "bank" has exactly one vector, so river-bank and money-bank are indistinguishable. Disambiguation happens later, inside attention.

EASY TO MISS

Embeddings are often multiplied by $\sqrt{d_{\text{model}}}$ before the positional signal is added, so the position information does not overwhelm the meaning. Also, the embedding matrix is frequently reused (transposed) as the final output projection — one matrix, two jobs, a large parameter saving.

4 Positional encodings

Attention computes a weighted sum, and sums do not care about order. Without a position signal a sentence collapses into a bag of words: the same words in any arrangement produce the same output.

$$X_i = E_i + PE_i$$

The original scheme uses fixed sine and cosine waves of different frequencies. For position k and dimension pair $(2i, 2i + 1)$ of an embedding of size d :

$$PE(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right), \quad PE(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

with $n = 10000$ in the original work. Each value of i contributes one sine–cosine pair at its own frequency, so the full vector is a unique multi-frequency signature of the position — an analogue version of a binary counter.

With $d = 4$ and $n = 100$, the first four positions are:

Position k	dim 0: $\sin(k)$	dim 1: $\cos(k)$	dim 2: $\sin(k/100)$	dim 3: $\cos(k/100)$
0	0.00	1.00	0.00	1.00
1	0.84	0.54	0.01	1.00
2	0.91	-0.42	0.02	1.00

3	0.14	-0.99	0.03	1.00
---	------	-------	------	------

Low dimensions turn fast (they separate neighbouring positions), high dimensions turn slowly (they encode coarse location in a long document).

Why sine and cosine specifically

Because a shift of k positions becomes a fixed linear transformation of the encoding:

$$\sin(pos + k) = \sin(pos) \cos(k) + \cos(pos) \sin(k)$$

So "three tokens back" is one learnable operation that works at every absolute position, rather than something the model must memorise separately for position 5 and position 500.

EASY TO MISS

The position signal is **added** to the embedding, not concatenated. Meaning and position therefore share the same dimensions and the model has to learn to separate them — which is exactly the weakness that rotary embeddings (next section) remove.

5 Rotary positional embedding (RoPE)

Instead of adding a position vector to the input, rotate the query and key vectors by an angle proportional to their position.

$$Q'_i = R_i Q_i, \quad K'_j = R_j K_j, \quad R(\theta_p) = \begin{bmatrix} \cos \theta_p & -\sin \theta_p \\ \sin \theta_p & \cos \theta_p \end{bmatrix}$$

The vector is split into consecutive pairs $(x_1, x_2), (x_3, x_4), \dots$ and each pair is rotated in its own 2-D plane, with a different frequency per pair — the same fast/slow idea as the sinusoids.

EXAMPLE

Take $Q = [1, 0]$ and a rotation of 90° per position. Position 0 gives $[1, 0]$, position 1 gives $[0, 1]$, position 2 gives $[-1, 0]$. Same word, same starting vector, different direction at each position.

The payoff appears in the attention score. Rotation matrices satisfy $R_i^\top R_j = R_{j-i}$, so:

$$(Q'_i)^\top K'_j = Q_i^\top R_{j-i} K_j$$

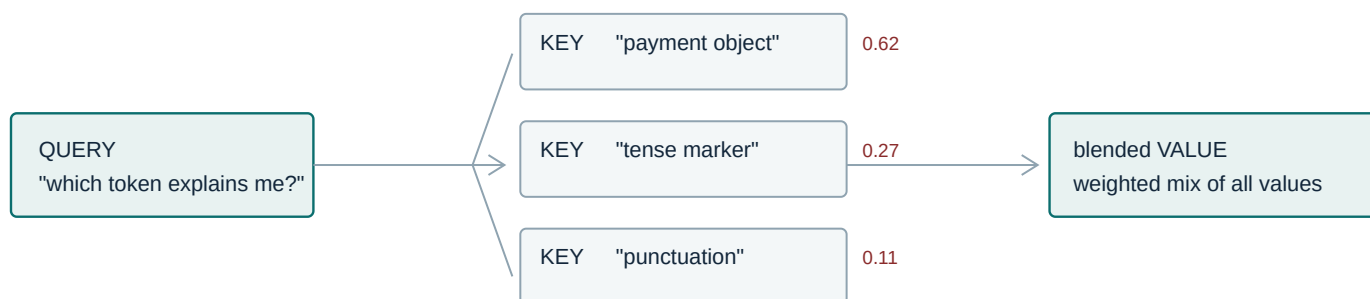
Key result. The score depends only on the **relative** distance $j - i$, never on the absolute positions. Position information enters the comparison instead of contaminating the content vector.

EASY TO MISS

Rotation preserves vector length, so RoPE changes direction without changing magnitude — the token's meaning survives intact. Because only relative offsets matter, the rotation frequencies can be rescaled after training to stretch a model trained at 8k context out to 128k. This is the practical machinery behind long-context releases, and it is why almost every current large model uses RoPE rather than additive encodings.

6 Query, key, value

Three learned linear projections of the same input: $Q = XW_Q$ (what I am looking for), $K = XW_K$ (what I advertise about myself), $V = XW_V$ (what I hand over if selected).



a dictionary returns one value for an exact key; attention returns a weighted blend of every value

Figure 2 — attention as a differentiable dictionary lookup.

EASY TO MISS

Q and K must live in the same space because they are dot-producted together; V need not, its width is a free choice. More importantly, **who supplies which** is the only structural difference between self-attention and cross-attention — same formula, different sources.

7 Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Read it as four steps, right to left:

1. **Score** — every query against every key: QK^T is an $n \times n$ matrix.
2. **Scale** — divide by $\sqrt{d_k}$.
3. **Normalise** — softmax along each row, so each row of weights sums to 1.
4. **Gather** — multiply by V : each output is a weighted sum of value vectors.

EASY TO MISS

The $\sqrt{d_k}$ is not cosmetic. The dot product of two random d -dimensional vectors has variance proportional to d , so for $d_k = 64$ the logits spread wide, the softmax saturates into a near one-hot, and the gradient through it collapses to almost zero. Dividing by $\sqrt{d_k}$ restores unit variance and keeps training alive.

8 Self-attention

Attention in which Q , K and V are all projections of the same sequence — tokens attending to their own neighbours. This is where a context-free embedding becomes context-aware.

EXAMPLE

"The technician replaced the drive because **it** was failing." The query from *it* matches the key of *drive* far more strongly than that of *technician*. Change "failing" to "careless" and the weights move to *technician*. Same weights, different input: reference resolved with no rule engine.

Shapes, concretely, for a sequence of length T : $Q, K, V \in \mathbb{R}^{T \times d_{\text{model}}}$, the score matrix is $\mathbb{R}^{T \times T}$, and the output Z is back to $\mathbb{R}^{T \times d_{\text{model}}}$.

EASY TO MISS

That $T \times T$ matrix is the whole cost story: doubling the context **quadruples** attention compute and memory. It also explains the KV cache — during generation the keys and values of past tokens are stored and reused rather than recomputed, and the size of that cache, not raw arithmetic, is usually what limits how many users a server can hold at once.

9 Multi-head attention

Run h attention operations in parallel on *slices* of the vector, concatenate the results, then apply one output projection.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O, \quad d_h = d_{\text{model}}/h$$

With $d_{\text{model}} = 512$ and $h = 8$, each head works in a 64-dimensional subspace. One softmax can only be confident about one relationship at a time; several heads let several relationships be tracked at once.

EXAMPLE

"The engineer filed the report because she finished it." Useful relations here: *she* $\rightarrow$ *engineer* (reference), *finished* $\rightarrow$ *report* (semantic), *filed* $\rightarrow$ *report* (action and object). Different heads can carry different ones.

Common misconception. Heads are not assigned roles such as "one head for grammar, one for semantics". Each head simply learns its own attention pattern and its own subspace. Interpretable roles sometimes emerge, but they are a by-product, not a design.

EASY TO MISS

Multi-head is essentially free: dimensions are **split, not duplicated**, so changing h leaves the parameter count unchanged as long as d_{model} is fixed. Serving systems now go further with grouped-query attention, where many query heads share one key/value head purely to shrink the KV cache.

10 Add & norm

Add = residual connection, **out** = $x + \text{Sublayer}(x)$. Norm = normalising each vector to mean 0 and standard deviation 1.

- The residual gives gradients a direct path to the input, which is what makes depth beyond a few layers trainable at all.
- It also forces each sublayer to learn an **edit** to a running representation rather than a replacement for it.
- Normalisation keeps activation scales stable across layers and acts as a mild regulariser.

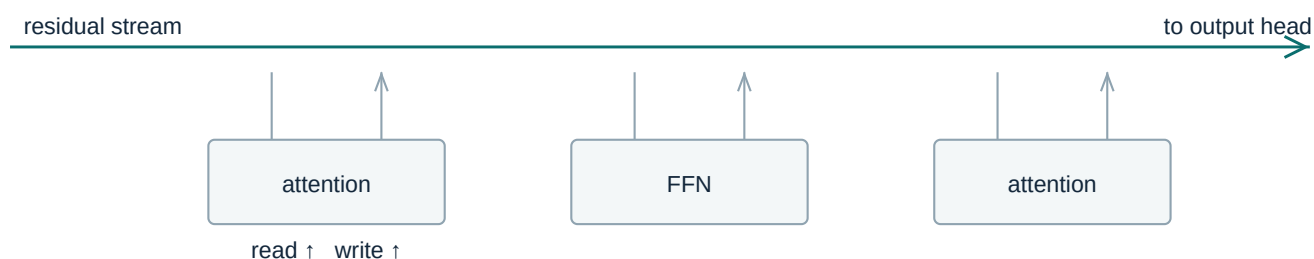


Figure 3 — the residual stream is a shared bus that every sublayer reads from and adds to.

EASY TO MISS

The classic diagram places the norm **after** the addition (post-norm), which needs a careful learning-rate warm-up or training diverges. Current models put the norm **before** the sublayer (pre-norm), leaving the residual path untouched; most have also replaced full layer normalisation with a scale-only variant that skips mean subtraction and is cheaper with no loss in quality.

11 Feed-forward network

A two-layer perceptron applied to each position independently and identically: expand, apply a non-linearity, contract.

$$h' = W_2 \sigma(W_1 h + b_1) + b_2, \quad d_{\text{model}} \rightarrow d_{\text{ff}} \rightarrow d_{\text{model}}, \quad d_{\text{ff}} \approx 4 d_{\text{model}}$$

What it contributes:

- **Feature mixing** — recombines the dimensions of one token's vector into new features.
- **Non-linearity** — without $\sigma(\cdot)$ the stacked linear layers would collapse into a single linear map.
- **Conditional logic** — a feature can be switched on only when another is present ("A matters when B is present").

	Self-attention	Feed-forward network
Main role	Exchange information across tokens	Transform one token's representation
Interaction	Token to token	Within a single token
Main operation	Weighted aggregation	Non-linear transformation
Depends on	The other tokens present	Only the current vector
Slogan	"Who should I listen to?"	"What do I do with what I heard?"

EASY TO MISS

This is the ordinary multi-layer perceptron, run once per token with shared weights — and it holds roughly **two thirds of all parameters**. There is good evidence it behaves as a key–value memory: the first matrix detects input patterns, the second writes out associated content, which is why factual-editing research targets these weights. Modern stacks replace the simple non-linearity with a gated variant, and mixture-of-experts models replace this single network with many, routing each token to one or two.

12 Where the parameters actually sit

Attention has four square projection matrices W_Q, W_K, W_V, W_O , each about d^2 ; the feed-forward network has two matrices of $d \times 4d$.

$$N_{\text{attention}} \approx 4d^2, \quad N_{\text{FFN}} \approx 2 d_{\text{model}} d_{\text{ff}} \approx 8d^2$$

Scale	d_{model}	d_{ff}	Attention / layer	FFN / layer
Small	768	3072	~2.4 M	~4.7 M
Medium	1024	4096	~4.2 M	~8.4 M
Large	4096	16384	~67 M	~134 M
Very large	8192	32768	~268 M	~537 M

Rule of thumb. Feed-forward parameters $\approx 2 \times$ attention parameters. Gated feed-forward networks and grouped-query attention shift the exact ratio, but the ordering does not change.

13 Encoder stack

N identical blocks (six in the original), each being self-attention $\rightarrow$ add & norm $\rightarrow$ feed-forward $\rightarrow$ add & norm. The output of block i is the input to block $i + 1$. No masking is used, so every token sees the entire input on both sides.

OUTPUT

Not a prediction — one context-rich vector per input token. That is what classification, named-entity tagging, retrieval embeddings and rerankers consume.

EASY TO MISS

Bidirectionality is precisely why an encoder **cannot** generate text left to right, and why encoder-style models are trained by masking out tokens and predicting them instead. It is also why encoders remain the better choice for building a retrieval index: for a document chunk you want the whole chunk visible at once.

14 Targets and "shifted right"

On the decoder side the target sequence is tokenised, embedded and given positional information in exactly the same way, then offset by one: the decoder input is `<sos>` followed by the target tokens, and the training label at each position is the *next* target token.

EXAMPLE

Target "the payment cleared" becomes input `<sos>` the payment cleared with labels the payment cleared `<eos>`. Position 2 sees "<sos> the" and must produce "payment".

EASY TO MISS

Feeding the *correct* previous tokens during training is called teacher forcing, and it creates a gap: the model trains on a perfect history but at generation time consumes its own possibly-wrong output, so one bad token can compound. Closing that gap is a large part of why post-training stages exist.

15 Masked attention

Self-attention with future positions blocked, so position i can only attend to positions $\leq i$. Implemented by adding a mask matrix M to the scores before the softmax, with $-\infty$ above the diagonal.

$$\text{MaskedAttention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

EASY TO MISS

Two details. First, the mask must be $-\infty$ **added to the logits**, not zeros applied to the weights afterwards — otherwise the softmax denominator would still include the future tokens and leak information. Second, the mask is what allows a whole sentence to be trained in **one parallel pass**: every position predicts its own next token simultaneously without cheating.

16 Encoder–decoder (cross) attention

Keys and values come from the encoder output; queries come from the decoder. One line of difference from self-attention, and it is the bridge between the two towers.

EXAMPLE

While generating the third word of a translation, the decoder's query searches the encoder's keys and lands on the source word it is translating. Plotting those weights recovers a word alignment map that used to need a dedicated model.

EASY TO MISS

No causal mask is used here — the source is not being predicted, so the decoder may look at all of it. The encoder's keys and values are computed **once** and reused at every decoding step; only the query changes. The same mechanism lets text queries attend to image patches in multimodal models.

17 Decoder stack, linear, softmax

The decoder block has **three** sublayers rather than the encoder's two: masked self-attention, then cross-attention, then the feed-forward network, each wrapped in add & norm. After N blocks:

- **Linear** — one projection from d_{model} to the vocabulary size V , producing raw logits, one score per possible next token. At $d = 4096$ and $V = 128,000$ this single matrix holds roughly 500 M parameters, which is why it is often tied to the input embedding.
- **Softmax** — turns the logits into a probability distribution; training pairs it with cross-entropy loss against the true next token.

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

The temperature T and the sampling rule (top- k , top- p) live entirely at this last step: same weights, very different behaviour.

EASY TO MISS

At generation time only the **last** position's vector is projected to the vocabulary; the other $T - 1$ vectors are discarded, having existed only to be attended to. During training all positions are projected, since every position is a supervised prediction. That asymmetry is the main reason training and inference have such different cost profiles. Note also that softmax always returns a valid distribution — the model has no structural way to say "I do not know" at this layer, which is part of why confident errors occur.

18 The same machinery beyond text

- **Vision** — split the image into fixed patches (16×16 pixels), flatten and linearly project each patch to get "patch embeddings", add learnable positional embeddings, prepend a learnable classification token, and feed the sequence into a standard encoder. The classification token's final vector goes to a small head.
- **Audio** — the same idea over spectrogram frames instead of image patches; encoder-only variants with a sequence-alignment loss, or encoder–decoder variants for transcription. Convolutional front-ends are often added.
- **Multimodal** — separate encoders per modality, a shared decoder with per-task query embeddings, and small task-specific heads, so one network covers detection, visual question answering and text classification at once.

EASY TO MISS

Nothing in the block is language-specific. Anything that can be cut into a sequence of vectors — patches, audio frames, sensor readings, time series — can be fed to the identical component stack. The modality only changes the tokenizer and the head.

19 Worked example: one sentence, every stage

A deliberately tiny model, so every number can be checked by hand: $d_{\text{model}} = 4$, one head, one block, vocabulary of six words. The prompt is "the expired card was" and the task is to predict the next token. This is the decoder-only setting used by current language models — no encoder, no cross-attention.

STAGE 1 — TOKENIZATION

The sentence is split and mapped to ids. Here each word happens to be one token.

the	expired	card	was	→	[41]	[3092]	[712]	[88]
-----	---------	------	-----	---	------	--------	-------	------

Sequence length $T = 4$. From here on, position indices are 0, 1, 2, 3.

STAGE 2 — EMBEDDING LOOKUP

Each id indexes a row of the embedding table, and the result is scaled by $\sqrt{d_{\text{model}}} = 2$.

Token	dim 0	dim 1	dim 2	dim 3	Loosely, what the dimensions carry
the	0.20	0.00	0.20	0.00	dim 0: "state / validity" dim 1: "payment object" dim 2: "function word" dim 3: "negative outcome"
expired	1.80	0.40	0.00	1.60	
card	0.40	1.80	1.40	0.20	
was	0.70	1.70	0.20	0.40	

Real embeddings are not interpretable dimension by dimension; the labels here are a teaching device.

STAGE 3 — ADD POSITIONAL ENCODING

Using the sinusoidal table from section 4 with $d = 4$, $n = 100$: $X = E + PE$.

Token (position)	E	PE	$X = E + PE$
the (0)	[0.20, 0.00, 0.20, 0.00]	[0.00, 1.00, 0.00, 1.00]	[0.20, 1.00, 0.20, 1.00]
expired (1)	[1.80, 0.40, 0.00, 1.60]	[0.84, 0.54, 0.01, 1.00]	[2.64, 0.94, 0.01, 2.60]
card (2)	[0.40, 1.80, 1.40, 0.20]	[0.91, -0.42, 0.02, 1.00]	[1.31, 1.38, 1.42, 1.20]
was (3)	[0.70, 1.70, 0.20, 0.40]	[0.14, -0.99, 0.03, 1.00]	[0.84, 0.71, 0.23, 1.40]

Notice that "the" at position 0 and a hypothetical "the" at position 3 would now have different vectors — meaning plus position, as intended.

STAGE 4 — PROJECT TO Q, K, V

Three linear maps. In this toy model W_Q and W_K keep the two semantic dimensions at full weight and halve the rest, and W_V is the identity, so $V = X$.

Token	$Q = XW_Q$	$K = XW_K$	$V = XW_V$
the	[0.20, 1.00, 0.10, 0.50]	[0.20, 1.00, 0.10, 0.50]	[0.20, 1.00, 0.20, 1.00]
expired	[2.64, 0.94, 0.01, 1.30]	[2.64, 0.94, 0.01, 1.30]	[2.64, 0.94, 0.01, 2.60]
card	[1.31, 1.38, 0.71, 0.60]	[1.31, 1.38, 0.71, 0.60]	[1.31, 1.38, 1.42, 1.20]
was	[0.84, 0.71, 0.12, 0.70]	[0.84, 0.71, 0.12, 0.70]	[0.84, 0.71, 0.23, 1.40]

STAGE 5 — SCORES, MASK, SOFTMAX

$S = QK^T/\sqrt{4}$. Rows are queries, columns are keys.

	the	expired	card	was
the	0.65	1.06	1.01	0.62
expired	1.06	4.78	2.77	1.90
card	1.01	2.77	2.25	1.29
was	0.62	1.90	1.29	0.86

Now add the causal mask — everything above the diagonal becomes $-\infty$ — and softmax each row. The greyed cells contribute exactly zero weight.

	the	expired	card	was	row reading
the	1.00	$-\infty$	$-\infty$	$-\infty$	first token can only see itself
expired	0.02	0.98	$-\infty$	$-\infty$	nothing informative behind it yet
card	0.10	0.57	0.34	$-\infty$	"card" absorbs "expired"
was	0.13	0.46	0.25	0.16	last token pulls from both content words

This is the moment the sentence stops being a bag of words. The vector for "card" is now 57 % shaped by "expired" — an invalid card rather than a generic one.

STAGE 6 — GATHER VALUES, THEN ADD & NORM

$Z = AV$, then the residual $R = X + Z$, then normalise each row to mean 0 and standard deviation 1.

Token	Z (attention output)	$R = X + Z$	after norm
the	[0.20, 1.00, 0.20, 1.00]	[0.40, 2.00, 0.40, 2.00]	[-1.00, 1.00, -1.00, 1.00]
expired	[2.58, 0.94, 0.01, 2.56]	[5.22, 1.88, 0.02, 5.16]	[0.97, -0.54, -1.37, 0.94]
card	[1.96, 1.09, 0.50, 1.97]	[3.27, 2.48, 1.92, 3.17]	[1.02, -0.42, -1.44, 0.85]
was	[1.70, 1.02, 0.42, 1.85]	[2.55, 1.73, 0.65, 3.25]	[0.52, -0.32, -1.44, 1.25]

Compare the "was" row before and after: it entered the block as [0.84, 0.71, 0.23, 1.40], carrying almost nothing about the situation, and leaves carrying a strong dimension-3 component — the "negative outcome" direction it inherited from "expired" via "card".

STAGE 7 — FEED-FORWARD NETWORK

Only the last position matters for the next-token prediction, so follow the "was" vector $u = [0.52, -0.32, -1.44, 1.25]$ through $W_2 \text{ReLU}(W_1 u + b_1)$ with $d_{\text{ff}} = 8$. Four of the eight hidden units, with weights chosen so their roles are visible:

Hidden unit	What its weights detect	pre-activation	after ReLU
H1	payment object in an invalid state	1.29	1.29
H2	generic concrete noun	-0.26	0.00
H3	interrogative form	-1.07	0.00
H4	past-tense copula	1.05	1.05



Figure 4 — the non-linearity is a gate: two units carry information forward, two are switched off entirely.

The second matrix reads those two active units and writes their associated content back into the residual stream:

FFN output	$u + \text{FFN output}$	after norm
[1.09, -0.75, 0.47, 1.33]	[1.61, -1.07, -0.97, 2.58]	[0.67, -1.01, -0.95, 1.28]

Attention decided *who to listen to*; the feed-forward network decided *what that combination means*. In a real model this is where stored associations such as "expired payment instrument → declined" actually live.

STAGE 8 — LINEAR AND SOFTMAX

The final vector is multiplied by the output matrix (six rows here, one per vocabulary word) to give logits, then softmaxed.

Candidate token	logit	probability at $T = 1$	
declined	2.51		0.673
expired	1.40		0.220
charged	0.26		0.071
approved	-1.38		0.014
today	-1.42		0.013
the	-1.80		0.009

Sampling temperature reshapes the same six logits without touching a single weight:

Temperature	declined	expired	charged	others	Effect
$T = 0.5$	0.894	0.096	0.010	0.000	near-deterministic, repetitive
$T = 1.0$	0.673	0.220	0.071	0.036	the trained distribution
$T = 1.5$	0.526	0.250	0.117	0.107	more varied, less reliable

Take "declined", append it to the sequence, and the whole process repeats for position 4 — with the keys and values of the first four tokens read from cache rather than recomputed.

What the walk-through shows

1. Meaning enters as a static lookup, and stays static until attention runs.
2. The causal mask is why row 1 of the attention matrix is trivially [1, 0, 0, 0], and why the model can be trained on all four positions at once.
3. "card" absorbing "expired" (weight 0.57) is the entire contribution of self-attention in this sentence — one number doing the work of a coreference rule.
4. The feed-forward network converts that mixed representation into a prediction-shaped one; the residual stream carries everything forward untouched alongside it.
5. Only the final position is projected to the vocabulary. The other three vectors are discarded.

20 Recall check

1. Why divide the scores by $\sqrt{d_k}$?
 2. Why does removing positional information turn a sentence into a set?
 3. What property of rotation matrices makes RoPE relative?
 4. Where do most of the parameters live, and by what factor?
6. Why $-\infty$ before the softmax rather than zeroing weights after?
 7. What does the KV cache store, and why does it bound serving capacity?
 8. Post-norm or pre-norm: which is drawn in the classic diagram, which is used today?
 9. Does increasing the number of heads increase the parameter count?

5. Which of Q , K , V comes from where in cross-attention?

10. What happens to the other $T - 1$ output vectors at generation time?